

★ WEEK 13 · LECTURE

Learning to Act

Reinforcement learning — and smart trade execution

Expanded edition · 38 slides · with three hands-on labs

So far our models predicted. Now they ACT and learn from the results, like a puppy earning treats. We meet the trial-and-error loop and use it to trade more cheaply.

Where We Are Going Today

Prediction told us WHAT will happen. RL learns WHAT TO DO about it

1

The learning loop

Agent, action, reward. How an AI learns by trial and error — no answer key, just feedback.

2

Explore, exploit, and Q-learning

Balancing safe choices with risky discovery, and building the 'how-good-is-this-move' table.

3

Smart execution

Use RL to trade a big order without moving the price against yourself.

By the end you will price patience with γ , fill a Q-table by hand, and out-trade a naive schedule.

1

PART 1 · THE LOOP

The Learning Loop

Agent, action, reward

How You'd Train a Puppy

No instruction manual — just rewards and corrections

You don't hand a puppy a rulebook. It tries things; good behavior gets a treat, bad behavior doesn't. Over time it learns what earns treats. Reinforcement learning trains an AI the exact same way.

How it differs from before

- Supervised learning: copy the labeled answers.
- Unsupervised: find structure with no answers.
- RL: ACT, get feedback, and improve over time.

What makes it special

- It learns from CONSEQUENCES, not labels.
- Actions today affect rewards far in the future.
- Perfect for tasks that are a sequence of decisions.

Three Ways a Machine Can Learn

The course's whole journey, in one table — and where today sits

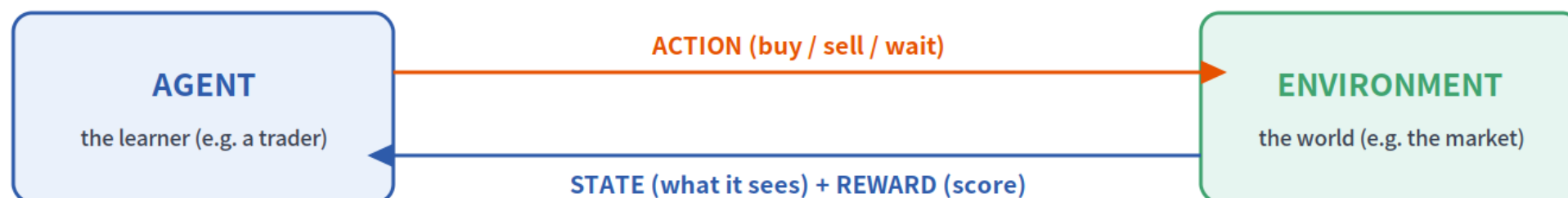
	Supervised (W3–W8)	Unsupervised (W9)	Reinforcement (today)
The feedback	the right answer, per row	none — find structure	a reward, after acting
The question	what WILL happen?	what IS in here?	what should I DO?
Finance example	will this loan default?	which stocks cluster?	how do I trade this order?
Hardest part	honest labels & testing	validating structure	the environment shifts

RL is the only one whose choices CHANGE the world it is learning about — markets included.

The RL Feedback Loop

Act on the world, observe the new state and a reward, then improve. Repeated millions of times until the agent gets good.

Reinforcement Learning — Learn by Trial and Reward



Like training a puppy

The agent acts, the world responds with a new situation and a reward (treat) or penalty (no treat).

Over many tries, it learns which actions earn the most reward over time. No answer key — just feedback.

What the Agent Is Chasing

Not today's reward — the TOTAL reward over time

$$\text{total reward} = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots$$

Reading it

- r_1, r_2, r_3 are the rewards at each step.
- We add up ALL of them — the agent plays the long game.
- γ (between 0 and 1) shrinks far-off rewards a bit.

Why the long game matters

- A small loss now can lead to a big win later — spend to learn, then profit.
- γ sets how patient the agent is.
- Small γ = greedy now; large γ = patient.

Pricing Patience by Hand

Grab (10, 0, 0) vs Invest (−2, 0, 20) — γ decides who wins (full version: problem set)

γ (patience)	Grab: $G = 10$	Invest: $G = -2 + \gamma^2 \cdot 20$	Winner
0.5 (greedy)	10	$-2 + 0.25 \cdot 20 = 3.0$	GRAB — the future barely counts
0.9 (patient)	10	$-2 + 0.81 \cdot 20 = 14.2$	INVEST — the future nearly full price
$\gamma^* = \sqrt{0.6} \approx 0.775$	10	exactly 10	the indifference point

One dial turned a sacrifice into a mistake and back. γ is the agent's time preference — set it to match the task's horizon.

2

PART 2 · EXPLORE & LEARN

Explore, Exploit, Learn

The dilemma and Q-learning

Explore vs. Exploit

Stick with what works, or risk trying something new? Too much exploit = stuck in a rut; too much explore = never settles.

Explore vs. Exploit — the Restaurant Dilemma

Exploit — go to your favorite



Pick the option you already know is good.

Safe, reliable reward.

But you might be missing something better.

Explore — try a new place



Take a risk on something untested.

Might discover a new favorite.

But it might be a letdown.

Good agents do both: mostly exploit, occasionally explore. That balance is the whole trick.

A Simple Rule for the Balance

'Epsilon-greedy': flip a weighted coin each step to decide explore or exploit

explore with prob. ε , exploit with prob. $1 - \varepsilon$

Reading it

- epsilon is a small number, say 10%.
- 10% of the time: try a random action (explore).
- 90% of the time: take the best known action (exploit).

The clever twist

- Start with big epsilon: explore a lot early on.
- Shrink it over time: exploit more as you learn.
- Like a kid who tries everything, then settles on favorites. Simple, but it works.

ϵ -Greedy Arithmetic on Two Brokers

Broker A has averaged 0.8, broker B 0.5. With $\epsilon = 0.2$, who gets the next order?

Path	Probability	Lands on A	Lands on B
Exploit ($1 - \epsilon = 0.8$)	0.8	0.8 (A is the best known)	0
Explore ($\epsilon = 0.2$, random pick)	0.2	$0.2 \times 0.5 = 0.1$	$0.2 \times 0.5 = 0.1$
Total	1.0	$P(A) = 0.9$	$P(B) = 0.1$

B still gets 10% of orders — the price of never being sure. If B is secretly better, only those visits will ever reveal it.

Q-Learning — the 'How Good Is This Move?' Table

For each situation, a score for every action. To act, read the current row and pick the highest — the hard part is filling it.

Q-Learning — a Cheat Sheet of "How Good Is This Move?"

Situation (state)	Buy	Sell	Wait
price falling	+8	-3	+1
price rising	-2	+6	0
choppy	-1	-1	+4

numbers = expected future reward (Q-value)

How it learns the table

Start with all zeros (it knows nothing).

Try an action, see the reward, nudge that cell up or down.

Repeat thousands of times until the numbers settle.

To act: read the row, pick the highest number. Simple.

How the Table Gets Filled In

Nudge each score toward the reward you actually saw

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Reading it (don't panic)

- $Q(s,a)$ = how good action a is in situation s .
- Take the reward r you got, plus the best you can do next.
- Compare to your old guess; nudge toward it — alpha sets the nudge size.

In plain words

- 'I expected this move to be worth 5. It turned out closer to 8.'
- 'So I'll bump my estimate up a little.'
- Repeat thousands of times and the table converges — Week 8's learn-from-error spirit.

Three Episodes, Filled In by Hand

1-D walk to a goal (states $0 \rightarrow 1 \rightarrow 2$), $\alpha = 0.5$, $\gamma = 0.9$, all Q start at 0 (full version: problem set)

After episode...	$Q(1, \text{right})$ — one step from the goal	$Q(0, \text{right})$ — two steps away
1	$0 + 0.5 \cdot (1 - 0) = 0.5$	$0 + 0.5 \cdot (0.9 \cdot 0 - 0) = 0$
2	$0.5 + 0.5 \cdot (1 - 0.5) = 0.75$	$0 + 0.5 \cdot (0.9 \cdot 0.5 - 0) = 0.225$
3	0.875	$0.225 + 0.5 \cdot (0.9 \cdot 0.75 - 0.225) = 0.45$

Watch the goal's glow spread BACKWARD one state per episode — that is temporal-difference learning, in four rows.

γ , α , ϵ — Three Dials, One Table

The course's U-curve, three more costumes

Dial	Too low	Too high	The habit
γ (patience)	greedy — sacrifices never pay	obsesses over a distant, noisy future	match the task's true horizon
α (nudge size)	learns glacially	each surprise erases old wisdom	start higher, decay as the table settles
ϵ (curiosity)	stuck in the first rut it found	never settles on anything	start high, shrink as knowledge grows

Same drama as h , λ , k , C , γ -kernel, ν , η and the bottleneck — every learner balances stability against adaptation.

3

PART 3 · SMART EXECUTION

Smart Execution

Trading a big order without moving the price

The Problem: You Are Big Enough to Matter

A huge order moves the very price you're trying to get

Want to buy a million shares? Slam it all in at once and the price jumps before you finish. You end up paying much more than the price you saw. This extra cost is called 'market impact'.

Why it happens

- A big buy order eats up all the cheap sell offers.
- To fill the rest, you reach for pricier ones.
- Your own demand pushes the price against you.

The goal

- Get the whole order done...
- ...at a price as close to fair as possible.
- This is the 'optimal execution' problem.

One Order, Three Bills

10,000 shares, quadratic impact, a gently rising market — same order, three prices

2,000

Cost of slamming it all
in at once — impact
alone

650

TWAP's bill: tiny
impact, but drift taxes
the waiting

600

A front-loaded tilt:
pays some impact to
dodge the drift

70%

Saved by the simplest
slicing — before any
intelligence

The RL agent's whole job: find the tilt between impact and drift, day by day, automatically.

Slice the Order Over Time

Many small trades barely move the price; one giant trade does. Equal slices = TWAP; RL learns a smarter schedule.

Optimal Execution — Don't Move the Price Against Yourself



Why RL Fits Execution Perfectly

It's a sequence of decisions with a clear reward

The setup as an RL problem

- State: shares left, time left, current price.
- Action: how much to trade right now.
- Reward: minus the cost (impact + price drift) — a perfect fit for the loop.

What the agent learns

- Trade faster when the price is favorable.
- Slow down when its own trading is moving the market.
- Beat naive equal-slicing (TWAP) on average; adapt to each day.

Big banks and brokers use exactly this kind of agent to execute large client orders.

Three Schedules, Priced by Hand

Impact $k \cdot t^2$ ($k = 0.00002$) + drift $0.01/\text{period}$ — 10,000 shares over 10 periods (full version: problem set)

Schedule	Impact $\Sigma k \cdot t^2$	Drift $\Sigma t \cdot i \cdot 0.01$	Total	Read
All at once ($10,000 \times 1$)	2,000	0	2,000	impact catastrophe
TWAP ($1,000 \times 10$)	200	450	650	calm, but drift taxes the wait
Front-half ($2,000 \times 5$, then 0)	400	200	600	pays impact to dodge drift — wins

Flip the drift negative and TWAP wins again — the best schedule depends on conditions, which is exactly why an adaptive agent earns its seat.

The Execution Desk's Menu

Three schedulers, one honesty column

Scheduler	The recipe	Adapts to...	The weakness
TWAP	equal slices by the clock	nothing — pure discipline	ignores volume and price entirely
VWAP	slice along the day's typical volume curve	average intraday liquidity	yesterday's curve, not today's tape
RL agent	state → action, learned from rewards	price, liquidity, its own footprint	needs a faithful simulator + W10 honesty

Discipline first, adaptation second: an RL agent that can't beat TWAP after costs goes back to the lab.

4

PART 4 · HANDS ON

Your Turn — Hands On

Train an agent, then slice a trade

Where RL Shows Up in Our Three Apps

Acting, not just predicting

Trading & execution

the clearest win

Slice large orders to cut market impact; decide when to trade fast vs slow. The safest RL win in finance.

today's capstone

Recommendation

explore/exploit live

Decide which item to show next; balance safe favorites vs new discoveries. Clicks are the reward.

the bandit from slide 12, at scale

Chatbots

RLHF

Reinforcement learning from human feedback rewards helpful, honest answers — how Week 12's LLMs get polished.

the loop, aimed at language

The common thread: a sequence of decisions, each shaped by the reward it earns.

RL in Markets — Handle With Care

Powerful, but harder than the demos make it look

RL shines in games with clear rules and unlimited practice. Markets have neither. They are noisy, ever-changing, and you can't replay reality — so naive RL often overfits the past.

The traps

- A great backtest may be luck (Week 10).
- The market 'environment' shifts under your feet.
- Real trading costs are easy to underestimate.

Where it genuinely helps

- Execution: a narrow, well-defined sub-problem.
- Simulated environments you can replay safely.
- As one tool among many, with honest testing.

Three Questions Before You Reach for RL

Answer honestly — two 'no's and you want a simpler tool

1 · Is it a SEQUENCE?

decisions that chain

One-shot predictions belong to Weeks 3–8. RL earns its complexity only when today's action changes tomorrow's options.

execution: yes · loan approval: no

2 · Is the reward CLEAN?

**measurable, prompt,
ungameable**

'Minus the trading cost' is clean.
'Long-run portfolio greatness' is vague, delayed, and easy to hack.

a fuzzy reward trains a con artist

3 · Can you REPLAY it?

a faithful simulator

Millions of practice runs need a world that can be reset. If the sim is wrong, the agent masters a fiction.

sim-to-real is where RL projects die

Execution scores yes-yes-yes — which is exactly why it is RL's beachhead in finance.

RL Meets the Rest of the Course

Today's loop is wired into everything we've built

Earlier week	What it contributes to RL	
W8 · Neural nets	replace the Q-table with a network → deep RL (the AlphaGo recipe)	
W10 · Honesty	the agent's backtest needs purged testing and a luck discount too	
W11 · Portfolios	execution is how HRP's target weights become actual positions	
W12 · LLMs	RLHF: the loop, pointed at language, made chatbots helpful	
W14 (next)	explaining WHY the agent acted — causality and SHAP	

Prediction, honesty, allocation, language — the loop is the connective tissue.

Which RL Piece? — A 60-Second Drill

Cover the right column, answer out loud, then check yourself

Scenario	Diagnosis / dial
Agent sacrifices now but never gets repaid	γ too high for the task's real horizon — or the sim lies
Q-table thrashes; yesterday's lessons vanish	α too high — each surprise erases old wisdom
Agent found one okay broker and never looks again	ϵ too low — the rut; keep a sliver of curiosity
Execution bot brilliant in sim, bleeding live	sim-to-real gap + W10's luck — audit the simulator
Order sits while a rising market runs away	all-impact thinking — price the drift, tilt the schedule

If you got 4 of 5, the agent-trainer's instinct of Week 13 is yours.

Getting This Week's Data

A tiny world built in the notebook — no download

A built-in toy environment

- RL needs an environment to act in, not a data table.
- So we build a tiny market simulator right in the lab.
- It rewards good actions and penalizes bad ones. No internet needed.

What we'll do

- Lab 1: Q-learning in a tiny grid — watch it learn.
- Lab 2: all-at-once vs sliced (TWAP) execution.
- All in plain numpy — the core ideas, no heavy RL library.

[CODE] `colab_setup.py` · tiny toy world (built inline)

```
import numpy as np
# a 1-D walk: reach the goal at the right end for a reward
N_STATES = 6          # positions 0..5; 5 is the goal
ACTIONS = ['left', 'right']
rng = np.random.default_rng(0)
```

Hands-On Lab 1 — Watch an Agent Learn

Q-learning in a tiny 1-D world

[CODE] colab_lab1.py · tiny toy world (built inline)

```
import numpy as np
Q = np.zeros((6, 2))          # states x [left, right]
a, g, eps = 0.1, 0.9, 0.2     # learn rate, gamma, explore

for episode in range(200):
    s = 0
    while s != 5:              # 5 = goal
        act = (rng.integers(2) if rng.random() < eps
               else Q[s].argmax())
        s2 = min(5, s+1) if act == 1 else max(0, s-1)
        r = 1 if s2 == 5 else 0
        Q[s, act] += a * (r + g*Q[s2].max() - Q[s, act])
        s = s2
print(Q.round(2))              # right should score highest
```

Your task (step by step)

- Start with an empty Q-table (all zeros).
- Let the agent wander (epsilon-greedy).
- Update Q after each step.
- After training, read the learned table.
- Q: did it learn to always head right?

Hands-On Lab 2 — All-at-Once vs. Sliced

See how slicing cuts the market-impact cost

[CODE] colab_lab2.py · tiny toy world (built inline)

```
import numpy as np
shares, impact = 10000, 0.0001 # price worsens with size

def cost(trades):
    # sum of size-squared impact
    return sum(impact * t**2 for t in trades)

all_at_once = [shares]
sliced = [shares/10] * 10 # TWAP: 10 equal slices

print('all at once cost:', round(cost(all_at_once), 1))
print('sliced (TWAP) cost:', round(cost(sliced), 1))
print('slicing saved:',
      round(cost(all_at_once) - cost(sliced), 1))
```

Your task (step by step)

- Model impact: bigger trade = worse price.
- Cost of dumping all shares at once.
- Cost of slicing into equal pieces (TWAP).
- Compare the two total costs.
- Q: how much does slicing save?

Hands-On Lab 3 — Claude as Your Coding Copilot

AI is allowed in this course — it explains and extends; you understand and verify

[INPUT] Prompt · Claude (paste after your Lab 1 + 2 code)

You are my machine-learning tutor.

- 1) Explain my Lab 1 and Lab 2 code above, one plain-English sentence per line.
- 2) Track Lab 1's learning: record steps-to-goal per episode and plot the learning curve; then rerun with ϵ in $[0.0, 0.2, 0.8]$ and overlay the three curves.
- 3) Verify my hand-calc: with $\alpha=0.5$, $\gamma=0.9$ on a 3-state walk going right each time, print Q after each of 3 episodes - expect $Q(1,R)$: $.5/.75/.875$ and $Q(0,R)$: $0/.225/.45$.
- 4) Execution: add drift $0.01/\text{period}$ and $k=0.00002$; price all-at-once, TWAP, front-half; then grid-search the best 10-slice schedule and plot it.
- 5) In 3 sentences: what the best schedule traded off, in this week's language.

House rules for AI use

- Allowed: explain, debug, extend code you wrote.
- Required: run it, read it, defend every line.
- Forbidden: submitting code you cannot explain.
- Tip: ask 'why', not just 'do'.

Comprehension Check

Try each question first — the answer is revealed below

Q1 In reinforcement learning, what does the agent learn from?

- A** Labeled answers
- B** Rewards and penalties from its actions ✓
- C** A fixed rulebook
- D** Nothing

Answer: RL learns from consequences — rewards and penalties — aiming to maximize total reward over time.

Q2 Why slice a big order into smaller trades over time?

- A** To trade faster
- B** To reduce market impact (moving the price against yourself) ✓
- C** To pay more fees
- D** To shuffle the data

Answer: A huge order moves the price against you; slicing keeps the market calm and the price near fair.

Comprehension Check — Round Two

Two more — these cover the new slides

Q3 At $\gamma = 0.5$, 'invest (-2 now, +20 later)' lost to 'grab 10 now'. Why?

- A** Investing is always worse
- B** γ^2 shrank the future payoff to 5 — patience was priced too low ✓
- C** The reward was negative
- D** α was too small

Answer: $20 \cdot \gamma^2 = 5$ at $\gamma = 0.5$: minus the upfront 2, invest scores 3 vs grab's 10. Above $\gamma \approx 0.775$ the order flips.

Q4 Your execution agent dominates in simulation but loses live. Most likely:

- A** γ is too small
- B** The simulator flatters it — sim-to-real gap (plus W10's luck) ✓
- C** The Q-table is too big
- D** TWAP is unbeatable

Answer: It mastered a fiction: unmodeled impact, costs, and regime shifts. Audit the sim; test with W10's honesty.

Week 13 on One Page

Photograph this slide — it is the whole lecture

Concept	One line to remember
The loop	state $\rightarrow$ action $\rightarrow$ reward $\rightarrow$ improve; the agent's choices change its world.
Return & γ	$G = r_1 + \gamma r_2 + \gamma^2 r_3 \dots$; γ prices patience — $\sqrt{\cdot}$ 0.6 flipped grab vs invest.
ϵ -greedy	mostly the best known move, ϵ of the time a random one; decay ϵ over time.
Q-learning	$Q \leftarrow Q + \alpha(r + \gamma \max Q' - Q)$ — the goal's glow spreads backward.
Execution	cost = impact ($k \cdot t^2$) + drift; slicing saved 70%, tilting saved more.
The caveat	markets shift and can't be replayed — narrow problems, honest tests (W10).

Next stop: from 'what predicts' to 'what CAUSES' — and explaining any model's decision.

Three Ideas, One Picture Each

RL shifts the question from 'what will happen?' to 'what should I do?' — handle markets with care

1

RL learns by trial and reward

Agent acts on the world, gets a reward, and improves. It chases TOTAL reward over time, not just now.

2

Explore, exploit, and Q-learning

Mostly take the best known action, sometimes try something new. Q-learning fills a 'how-good-is-this-move' table.

3

Smart execution

Slice a big order so it doesn't move the price. RL learns a schedule that beats naive equal-slicing (TWAP).

Keep the pictures: a puppy earning treats, a table filling backward — and an order melting into the tape.

Week 13 — Three Things to Remember

Before next class: finish Labs 1–3 and check how much slicing saved over dumping all at once

1

RL = act, get reward, improve

A different kind of learning: from consequences, not labels. The agent maximizes long-run reward (gamma).

2

Balance explore and exploit

Epsilon-greedy: mostly use what works, sometimes try new things. Q-learning builds an action-value table.

3

Optimal execution is RL's best fit

Slice a big order to cut market impact. A clean, well-defined problem where RL genuinely helps.

Next week — Causality, SHAP & Fairness: from 'what predicts' to 'what causes', and explaining models.